MODULE 12 · SECTION 2 - INGESTING AND PROCESSING THE DATA

Dataflow and Beam: How a Pipeline Really Runs

PCollections and transforms, joins, fusion, autoscaling, streaming modes, templates and connectors.

SECTION 2 of 5 EXAM WEIGHT 25% MODULE 12 of 45

admin@sthithapragna.com

You write the recipe. Dataflow decides the kitchen, the cooks and the order of the steps.

Six words to know

TERM	WHAT IT IS
Pipeline	The whole recipe, from sources to sinks
PCollection	A data set flowing between steps. Bounded or unbounded
PTransform	A step that turns PCollections into new ones
ParDo and DoFn	Your code, run on each element in parallel
GroupByKey, Combine	Bring elements with the same key together
Runner	What executes the recipe: Dataflow, Flink, Spark

HOW IT WORKS

PCollections are immutable. Every step makes a new one, which is what lets steps run in parallel.

A DoFn with a dead-letter output

```
1  class ParseReading(beam.DoFn):
2      def process(self, msg):
3          try:
4              r = json.loads(msg)
5              if "meter_id" not in r:
6                  raise ValueError("no meter_id")
7              yield r
8          except Exception as e:
9              yield beam.pvalue.TaggedOutput("bad", {"raw": msg, "error": str(e)})
10
11 out = msgs | "Parse" >> beam.ParDo(ParseReading()).with_outputs("bad", main="good")
12 out.good | "Raw" >> beam.io.WriteToBigQuery(RAW_TABLE)
13 out.bad | "Dead letter" >> beam.io.WriteToBigQuery(ERROR_TABLE)
```

9 Bad ones go to a tagged output, with the error, never dropped.

13 The dead-letter table, ready for investigation.

THE PATTERN

One bad message must never crash a streaming pipeline. Catch it, tag it, keep it.

Side input or CoGroupByKey?

SIDE INPUT

Broadcast a small lookup

- Copied to every worker
- Small and slowly changing
- No shuffle of the main data
- Refresh it on a schedule in streaming

COGROUPBYKEY

Join two big collections

- Both sides shuffled by key
- Large on both sides
- Costs a shuffle
- Both inputs need the same key

THE TIEBREAKER

A few thousand rows that rarely change: side input. Millions on both sides: CoGroupByKey.

Fusion: usually a gift, sometimes a trap

- Dataflow **fuses** neighbouring steps, so it never stores the data between them.
- The trap: a step that **fans out**, like one file name becoming millions of rows.
- Fused to a tiny input, the fan-out runs on **only a few workers**.
- Break the fusion with **Redistribute**, called **Reshuffle** in older material.

> REMEMBER
>
> Partial combining before a GroupByKey is automatic. Never hand-roll it.

How Dataflow decides how many workers

- Signals: average **CPU** and the pipeline's **parallelism**.
- **Batch**: on by default, re-checked every 30 seconds. Default cap 2,000 workers.
- **Streaming**: on by default only with **Streaming Engine**.
- Set **maxNumWorkers** for cost control. Make sure quota and the subnet can keep up.

> THE POINT
>
> Autoscaling can only add workers that have something to do. Parallelism is the real limit.

Why won't it scale?

SYMPTOM	LIKELY CAUSE	FIX
Few workers, stuck reading	One large gzip file cannot be split	Many files, or uncompressed input
One worker at 100%, others idle	A hot key in a GroupByKey	Hot-key fan-out, or a better key
Fan-out step crawls	Fused to a tiny input	Redistribute after the fan-out
Stops at a fixed count	maxNumWorkers, quota or IP range	Raise the limit that is binding

THE TRAP

Bigger machines rarely fix a parallelism problem. Find what cannot be split.

Exactly-once or at-least-once?

EXACTLY-ONCE

The default

- No records dropped or duplicated
- Right for counts, sums, money
- Higher cost and latency
- Works with every sink

AT-LEAST-ONCE

Cheaper and faster

- Nothing lost, a few duplicates
- Map-only copies, downstream dedupe
- Needs Streaming Engine
- BigQuery: Storage Write API at-least-once

THE TIEBREAKER

Aggregations need exactly-once. A copy into a table that is deduplicated later can save money with at-least-once.

Four features to recognise

FEATURE	WHAT IT DOES	SIGNAL
Streaming Engine	Moves execution off workers into the service	Smaller workers, smoother scaling
Dataflow Shuffle	Moves batch shuffles off workers	Faster batch scale-down
Dataflow Prime	Vertical autoscaling of worker memory	Out-of-memory errors
FlexRS	Delayed start, mostly preemptible VMs, discount	Batch, not time-critical, cheaper

THE TRAP

FlexRS can wait up to six hours to start. 'Must finish by 06:00' may rule it out.

Classic or Flex templates?

CLASSIC

A frozen graph

- The job graph is saved once
- Parameters need ValueProvider
- The shape cannot change at launch
- Older, still common

FLEX

A container that builds the graph

- Image in Artifact Registry
- Graph built at launch from parameters
- Steps can change by parameter
- Google's recommendation for new work

THE TIEBREAKER

Different steps for different parameters points to Flex. Google-provided templates cover common jobs.

Reading and writing everything

- **PubSubIO**, **BigQueryIO**, **BigtableIO**, **KafkaIO**, text and Avro files on Cloud Storage.
- **JdbcIO** for Cloud SQL and other databases. Partition reads, or one worker does it all.
- **Managed I/O**: Dataflow runs and upgrades the Iceberg, Kafka and BigQuery connectors.
- BigQueryIO prefers the **Storage Write API** for streaming writes.

> THE TRAP
>
> A JDBC read with no partitioning is a single-threaded query, whatever the worker count.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Small lookup, changes rarely	Side input
Fan-out runs on few workers	Redistribute, or Reshuffle
One worker hot in a grouping	Fix the hot key
Map-only, dedupe later, cut cost	At-least-once streaming mode
Out-of-memory errors on workers	Dataflow Prime vertical autoscaling
Steps vary by launch parameter	Flex template

THE PATTERN

Most Dataflow questions are about parallelism, state, or who launches the job.

> A Larkspur batch job reads a small manifest listing 2,000 exported history files, then expands each file name into millions of readings.
>
> maxNumWorkers is 100, but the job sits on two busy workers for hours while the rest stay idle.

What should the team change?

A Raise maxNumWorkers from 100 to 500, to give it more room

B Switch the workers to a larger machine type with more memory

C Add a Redistribute step right after the fan-out

D Run the job in streaming mode instead of batch

◆ PAUSE AND DECIDE

What should the team change?

A **A higher worker cap**

The cap was never reached. More room does not create parallel work.

B **Bigger machines**

Two bigger workers are still only two workers.

C **Redistribute**

Breaks the fusion, so millions of rows spread across every worker.

D **Streaming mode**

The data is bounded. The fusion problem would remain.

THE SIGNAL

A tiny input fanning out into millions of rows, on few workers, is fusion. Redistribute breaks it.

> Larkspur's streaming pipeline must add a region name to every reading, from a postcode area lookup of about 3,000 rows.
>
> The lookup changes a few times a year. The stream peaks at 10,000 readings a second.

How should the pipeline do the lookup?

A Load the lookup as a side input, and refresh it on a schedule

B Join the stream to the lookup table with a CoGroupByKey step

C Query Cloud SQL from inside the DoFn, once for every reading

D Write readings to BigQuery, and join them later in a nightly batch

◆ PAUSE AND DECIDE

How should the pipeline do the lookup?

A A refreshed side input

Small, slow-changing data on every worker, with no shuffle.

B CoGroupByKey

Shuffles ten thousand readings a second to join three thousand rows.

C Cloud SQL per reading

Ten thousand queries a second against a database.

D A nightly batch join

The region is needed now, not tomorrow.

THE SIGNAL

Small, rarely changing, needed on every element: a side input.

> A Larkspur pipeline copies readings from Pub/Sub into raw.meter_readings with no aggregation. Duplicates are removed later, when the curated table is built.
>
> The finance director wants the pipeline's cost and latency down.

What should the team change?

A Keep exactly-once mode, and add more workers at peak times

B Switch the job to Dataflow Prime for vertical autoscaling

C Write to BigQuery with nightly file load jobs instead of streaming

D Use at-least-once mode, with the Storage Write API at-least-once

◆ PAUSE AND DECIDE

What should the team change?

A More workers

Adds cost, the opposite of the goal.

B Dataflow Prime

Prime fixes memory problems. Nothing here is short of memory.

C Nightly file loads

Cheaper, perhaps, but a day of latency.

D At-least-once mode

Map-only, deduplicated later: the mode built for this, cheaper and faster.

THE SIGNAL

Map-only copy plus downstream dedupe plus cost pressure points to at-least-once mode.

> Larkspur's operations team launches the readings pipeline for the UK and for Germany, with different parameters. For Germany, the pipeline must add an extra tokenisation step.
>
> Operators have no development tools, and must not need the source code.

How should the pipeline be packaged?

A. A classic template, with ValueProvider parameters for each country

B. A Flex template whose image builds the graph from its parameters

C. The source code in a shared repository, run locally by operators

D. Two classic templates, one per country, kept in step by hand

◆ PAUSE AND DECIDE

How should the pipeline be packaged?

A One classic template

Its graph is frozen, so it cannot add a step for Germany.

B A Flex template

The graph is built at launch, so the German step appears when needed.

C Run from source

Operators have no tools, and must not need the code.

D Two classic templates

Two copies to keep in step, and Google recommends Flex for new work.

THE SIGNAL

'Different steps per parameter' plus 'no source code' is a Flex template.

Module 12 in five lines

- Beam is the recipe, Dataflow the kitchen. **Parallelism** is what really scales.
- **Side input** for small lookups. **CoGroupByKey** when both sides are big.
- Fan-out on few workers is **fusion**: Redistribute. One hot worker is a **hot key**.
- **At-least-once** for map-only copies. **Prime** for memory. **FlexRS** for patient batches.
- **Flex templates** build the graph at launch. Partition every JDBC read.

COMING UP

Next: module 13, windows: fixed, sliding, session and global.